Account Billing — API Flow Diagrams

Base URL: https://account-billing.onrender.com · Auth headers: `X-Invoice-Secret` + `X-Shopify-Shop-Domain`

Shop: **account-billing.myshopify.com** Customer storefront: **/pages/my-account** Document date: **2026-10-14**

☐ Start / Auth ☐ Webhook (ERP push) ☐ Database ☐ Success (2xx) ☐ Partial (202) ☐ Error (4xx)

0. System overview

ERP / Back-office
Webhooks push invoices & balances
→
Account Billing API
Render · Express app
→
PostgreSQL
Invoices · revisions · outstanding
→
Shopify Customer
My Account · View / PDF

#	Method	Endpoint	Type	Auth	Purpose
1	GET	`/api/health`	Health	None	App up check
2	GET	`/api/health/db`	Health	None	Database up check
3	POST	`/api/invoices/ingest`	**Webhook**	Required	Create invoice revision
4	GET	`/api/invoices/exists`	Query	Required	Check invoice exists
5	POST	`/api/outstanding/ingest`	**Webhook**	Required	Set customer outstanding balance
6	GET	`/api/outstanding`	Query	Required	Read customer outstanding balance

1. Health check — GET /api/health

Caller
Any client
→
GET /api/health
No auth
→
200 OK
status: ok

Payload

```
None (GET — no body)
```

Response — 200

```
{
  "success": true,
  "status": "ok",
  "service": "account-billing",
  "time": "2026-10-14T12:00:00.000Z"
}
```

2. Database health — GET /api/health/db



Payload

```
None (GET — no body)
```

Response — 200

```
{
  "success": true,
  "status": "ok",
  "database": "invoice_qhxr",
  "db_time": "2026-10-14T12:00:00.000Z",
  "latency_ms": 12
}
```

Payload

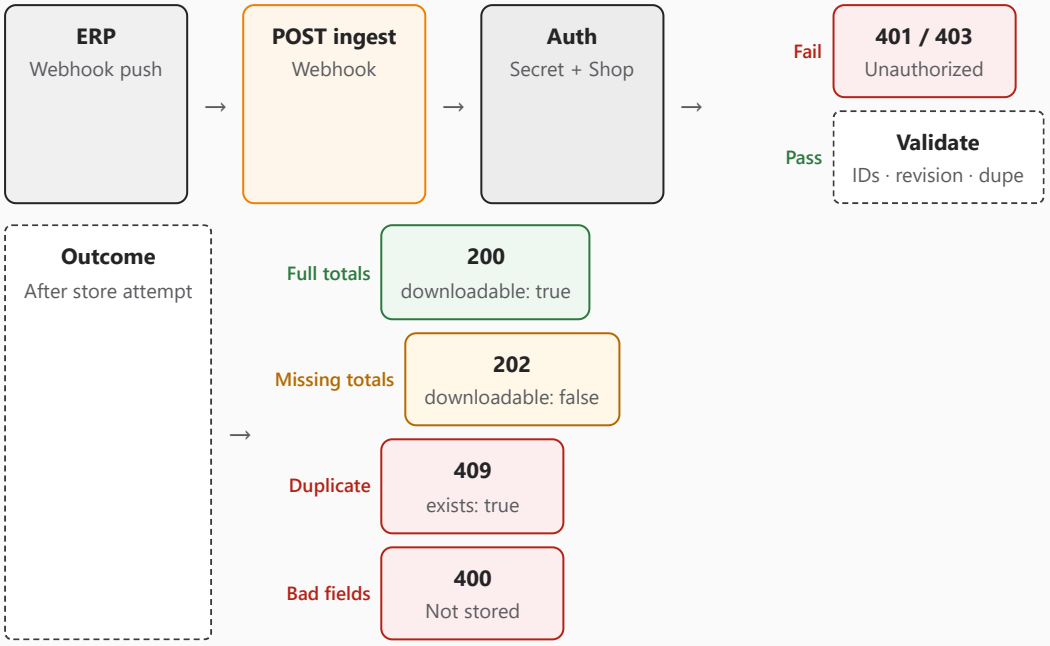
```
None
```

Response — 503

```
{
  "success": false,
  "status": "error",
  "error": "Database unavailable",
  "latency_ms": 45
}
```

3. Invoice ingest webhook — POST /api/invoices/ingest WEBHOOK

Webhook: ERP / back-office pushes one invoice (one revision) to this URL whenever an invoice is created or revised. Auth headers required on every call.



Webhook payload (request body)

POST body — example

```
{
  "shopify_customer_id": "31249944510693",
  "shopify_order_id": "5600000000999",
  "invoice_number": "IQ-LIVE-2001",
  "invoice_date": "2026-10-14",
  "order_no": "SO-46001",
  "order_date": "2026-10-13",
  "customer_no": "IQ-DEMO-01",
  "po_number": "PO-80001",
  "salesperson": "JD",
  "terms": "N30",
  "ship_via": "CANPAR",
  "currency": "CAD",
  "sold_to": {
    "name": "Customer Name",
    "address1": "100 Demo Street NW",
    "city": "Edmonton",
    "province": "AB",
    "postal": "T5L 4S9",
    "country": "CA"
  },
  "ship_to": {
    "name": "Ship To Name",
    "address1": "200 Ship Lane",
    "city": "Calgary",
    "province": "AB",
    "postal": "T2P 1J9",
    "country": "CA"
  },
  "lines": [
    {
      "item_number": "ITEM-001",
      "description": "Product description",
      "qty_ordered": 3,
      "qty_shipped": 3,
      "qty_backorder": 0,
      "unit_price": "12.50",
      "uom": "EACH",
      "extended_price": "37.50"
    },
```

```
    {
      "item_number": "FREIGHT",
      "description": "FREIGHT - CANPAR TRACK123",
      "qty_ordered": null,
      "qty_shipped": null,
      "qty_backorder": null,
      "unit_price": null,
      "uom": null,
      "extended_price": "18.50"
    }
  ],
  "tax_lines": [
    { "code": "GST", "amount": "2.80" },
    { "code": "PSTSK", "amount": "0.00" }
  ],
  "tax_exempt_code": "",
  "tax_registration_no": "123456789RT0001",
  "subtotal": "56.00",
  "total_tax": "2.80",
  "total_amount": "58.80",
  "less_payment": "0.00",
  "amount_due": "58.80",
  "revision": 1
}
```

Response — 200 (created, PDF OK)

```
{
  "success": true,
  "exists": false,
  "data": {
    "id": "...",
    "invoice_number": "IQ-LIVE-2001",
    "revision": 1,
    "downloadable": true,
    "missing_required_totals": []
  }
}
```

Response — 202 (stored, PDF off)

```
{
  "success": true,
  "exists": false,
  "data": {
    "downloadable": false,
    "missing_required_totals": ["total_amount", "amount_due"]
  },
  "warning": "Stored but not downloadable; missing required totals: ..."
}
```

Response — 409 (duplicate revision)

```
{
  "success": false,
  "exists": true,
  "error": "Invoice already exists for this invoice_number and revision",
  "data": {
    "exists": true,
    "invoice_number": "IQ-LIVE-2001",
    "revision": 1,
    "shopify_customer_id": "31249944510693"
  }
}
```

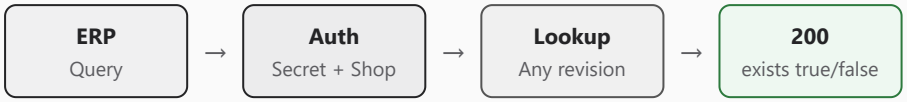
Response — 400 (required field empty)

```
{
  "success": false,
  "error": "invoice_number is required"
}

// also for empty shopify_customer_id
// or missing / invalid revision
```

Headers: **Content-Type: application/json**, **X-Invoice-Secret**, **X-Shopify-Shop-Domain**

4. Invoice exists — GET /api/invoices/exists



Payload (query or JSON body)

```
{
  "shopify_customer_id": "31249944510693",
  "invoice_number": "IQ-LIVE-2001"
}
```

Response — 200 (exists)

```
{
  "success": true,
  "data": {
    "exists": true,
    "shopify_customer_id": "31249944510693",
    "invoice_number": "IQ-LIVE-2001",
    "revision": 2,
    "invoice_date": "2026-10-14",
    "downloadable": true,
    "id": "..."
  }
}
```

Same payload

```
{
  "shopify_customer_id": "31249944510693",
  "invoice_number": "DOES-NOT-EXIST"
}
```

Response — 200 (not found)

```
{
  "success": true,
  "data": {
    "exists": false,
    "shopify_customer_id": "31249944510693",
    "invoice_number": "DOES-NOT-EXIST"
  }
}
```

5. Outstanding balance webhook — POST /api/outstanding/ingest WEBHOOK

Webhook: ERP / back-office pushes the customer's current outstanding balance to this URL. One balance per customer (overwrites previous value). Auth headers required.



Webhook payload (request body)

```
{
  "shopify_customer_id": "31249944510693",
  "outstanding_balance": 2600
}

// optional:
// "currency": "CAD"
```

Response — 200

```
{
  "success": true,
  "data": {
    "id": "...",
    "scope": "customer",
    "shopify_customer_id": "31249944510693",
    "amount": "2600",
    "currency": "CAD"
  }
}
```

Invalid payload example

```
{
  "shopify_customer_id": "",
  "outstanding_balance": null
}
```

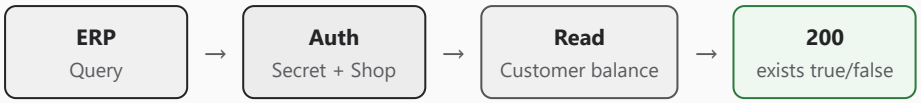
Response — 400

```
{
  "error": "shopify_customer_id is required"
}

// or outstanding_balance missing
```

Headers: **Content-Type: application/json**, **X-Invoice-Secret**, **X-Shopify-Shop-Domain**

6. Get outstanding — GET /api/outstanding



Payload (query or JSON body)

```
{
  "shopify_customer_id": "31249944510693"
}
```

Response — 200 (found)

```
{
  "success": true,
  "data": {
    "shopify_customer_id": "31249944510693",
    "exists": true,
    "outstanding_balance": "2600",
    "currency": "CAD"
  }
}
```

Same shape — unknown customer

```
{
  "shopify_customer_id": "99999999999999"
}
```

Response — 200 (none on file)

```
{
  "success": true,
  "data": {
    "shopify_customer_id": "99999999999999",
    "exists": false,
    "outstanding_balance": null,
    "currency": "CAD"
  }
}
```

7. Customer storefront (reference — not ERP)



ERP does not call these endpoints. Auth is the logged-in Shopify customer via app proxy.

8. Empty field policy (invoice webhook payload)

Field group	If empty / null / missing	Result
`shopify_customer_id` , `invoice_number` , `revision`	Rejected	**400** — not stored
`subtotal` , `total_tax` , `total_amount` , `less_payment` , `amount_due`	Still stored	**202** — downloadable: false
PO, salesperson, addresses, order id, freight qty/price, etc.	Allowed	**200** — stored normally
Same `invoice_number` + same `revision` again	Rejected	**409** — exists: true